

09/12/25

WAVE

Speedy techniques

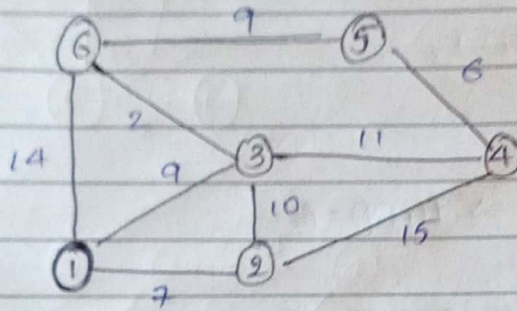
>> Dijkstra

>> Bell man ford.

>> Floyd

>> Marshall

① Dijkstra



$$d[u] + c(u,v) < d[v]$$

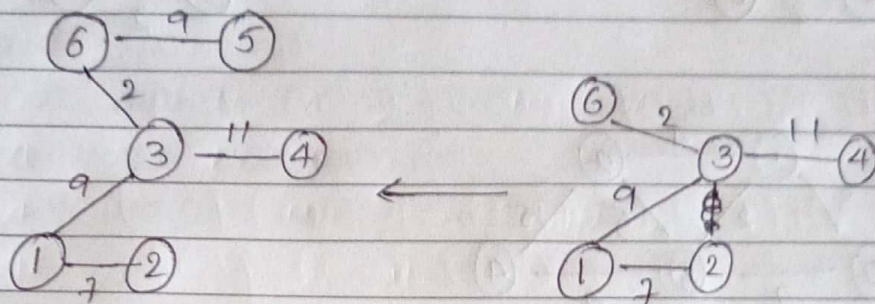
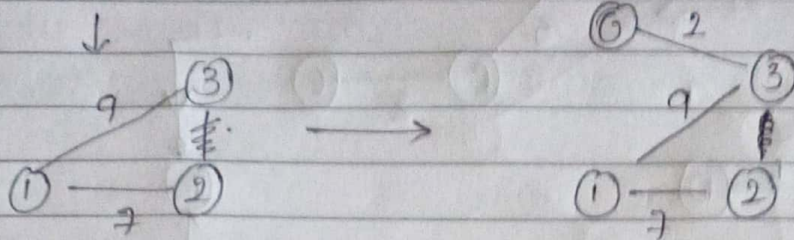
$$d[v] = d[u] + c(u,v)$$

single source shortest path.

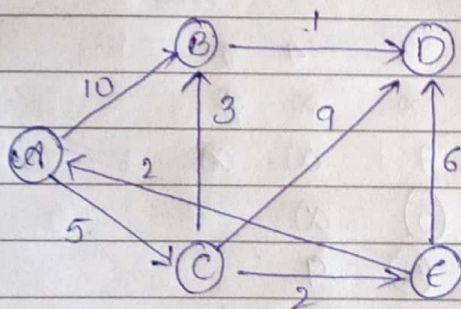
source	destination	1	2	3	4	5	6
1	2	∞	∞	∞	∞	∞	∞
1	3	9	∞	∞	∞	14	∞
1, 2	3	9	9	∞	∞	14	∞
1, 2, 3	3	9	9	10	∞	11	∞
1, 2, 3, 6	3	9	9	10	10	11	11
1, 2, 3, 6, 4	3	9	9	10	10	11	11
∴ 1, 2, 3, 6, 4, 5	3	9	9	10	10	11	11

→ ∴ 1, 2, 3, 6, 4, 5 //

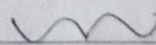
$\Rightarrow$ (1) — 7 — (2)



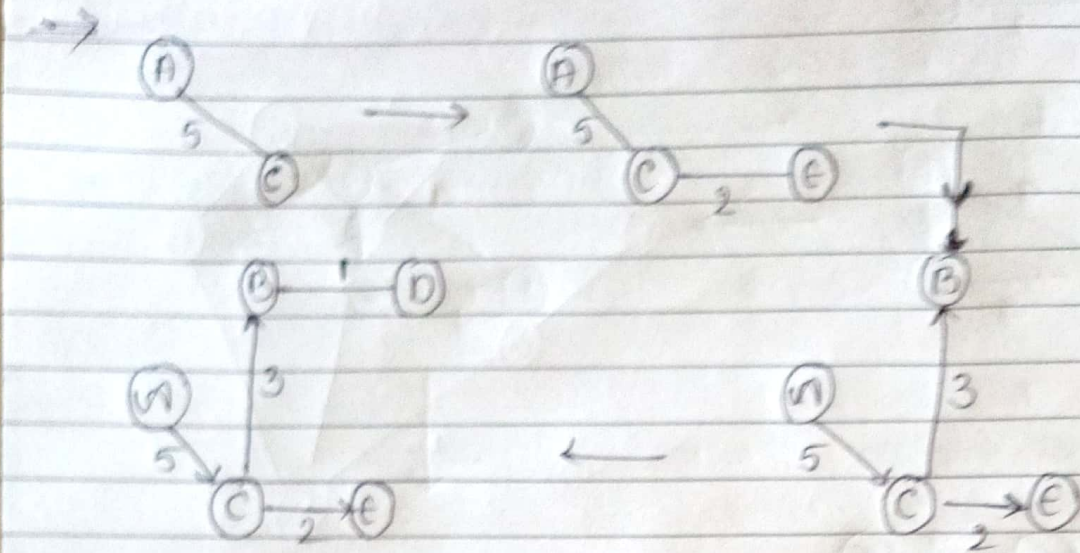
$\Rightarrow$ Ans:



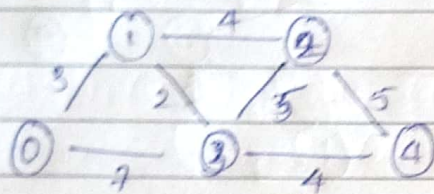
source	destination			
	B	C	D	E
A	∞	∞	∞	∞
A, C	10	(5)	∞	∞
A, C, E	8	(5)	14	(7)
A, C, E, B	(8)	(5)	13	(7)
	(8)	(5)	9	(7)



$\downarrow$
A, C, E, B, D.

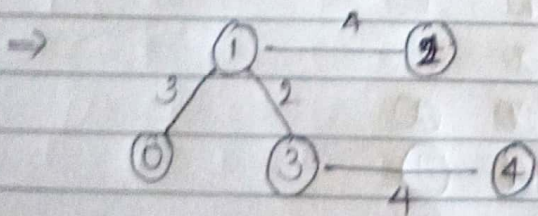


Ex:



Source	Destination	1	2	3	4	5
0		∞	∞	∞	∞	∞
0, 1		(3)	∞	7	∞	∞
0, 1, 3		(3)	7	(5)	∞	∞
0, 1, 3, 2		(3)	(7)	(5)	9	∞

→ 0, 1, 3, 2, 4



Algorithm:

// algo Dijkstra (G, src)

// input : weight graph.

// output : shortest dist from src to all other nodes.

start

for each vertex ' u '

$dist[u] = \infty$

$visited[u] = false$

$dist[s] = 0$

while there exist a un-visited vertex

$u =$ vertex with min dist.

$visited[u] = true;$

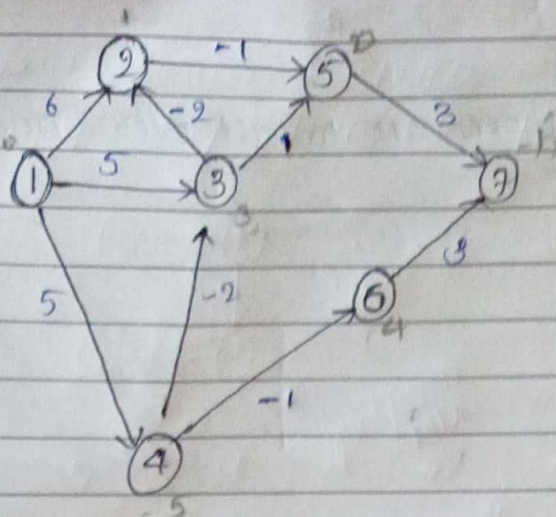
for each neighbour v of u

if $dist[u] + c(u, v) < dist[v]$

then $dist[v] = dist[u] + c(u, v)$

stop.

② Bellman ford:



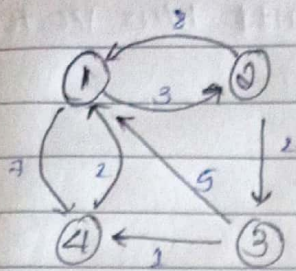
place in ascending order of vertices

(1,2) (1,3) (1,4) (2,5), (3,2) (3,5)
(4,3) (4,6) (5,7), (6,7).

→ compare the values of each node by following the order given above

<u>nodes</u> →	①	2	3	4	5	6	7
<u>values</u>							
pass ① :	0	3	3	5	4	5	7
pass ② :	0	1	3	5	3	4	5
pass ③ :	0	1	3	5	0	4	3
pass ④ :	0	1	3	5	0	4	3

③ Floyd: (all pair shortest path).



adjacency matrix

(i)

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

was intended

(ii)

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

(iii) $\downarrow$ (2) intermediate

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

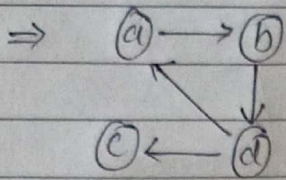
	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

$\downarrow$

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Marshall algo

>> it determines whether a path exists b/w vertex.
 >> By using "transitive property".



perform
AND.

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	1	1	0



	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1



	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	0	0	0	0
d	1	1	1	1